Problem A. Parantrees

Input file: `standard input`
Output file: `standard output`
Balloon Color: `Magenta`

Alice and Bob are playing a game on a tree with n nodes, rooted at node 1. Each edge of the tree has either an opening bracket '(' or a closing bracket ')' written on it.

Initially, the current node is the root, and the string s is empty. The players take turns, starting with Alice. In one move, the current player does the following:

- Choose an edge from the current node u to one of its children v.
- Append the bracket written on that edge to the end of s.
- Make v the current node.

The game ends when the current node is a leaf. If s is a regular bracket sequence, Alice wins; otherwise, Bob wins.

A bracket sequence is called regular if it can be transformed into a correct arithmetic expression by inserting characters 1 and + between its characters. For example, the sequences `()()` and `(())` are regular, while `)(`, `(` and `(()` are not.

Determine the winner if both players play optimally.

Input

The first line contains an integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($2 \le n \le 2 \cdot 10^5$) — the number of nodes in the tree.

The next $n-1$ lines contain two integers u and v and a character c ($1 \le u, v \le n$, $u \ne v$, c is either `(` or `)`) — an edge between nodes u and v with the bracket c written on it.

It is guaranteed that the given edges form a tree.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

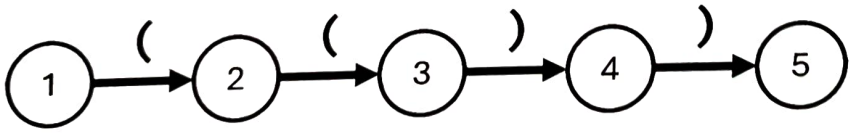
For each test case, output "Alice" if Alice wins; otherwise, output "Bob".

Example

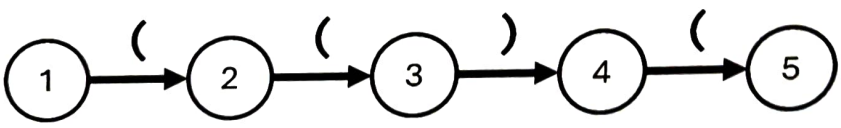
standard input	standard output
4 5 1 2 (2 3 (3 4) 4 5) 5 1 2 (2 3 (3 4) 4 5 (8 1 3 (3 4) 4 5 (2 1 (5 6) 2 7 (7 8) 9 1 2 (1 3 (3 4) 4 5 (5 6) 2 7 (7 8) 5 9 (	Alice Bob Alice Bob

Note

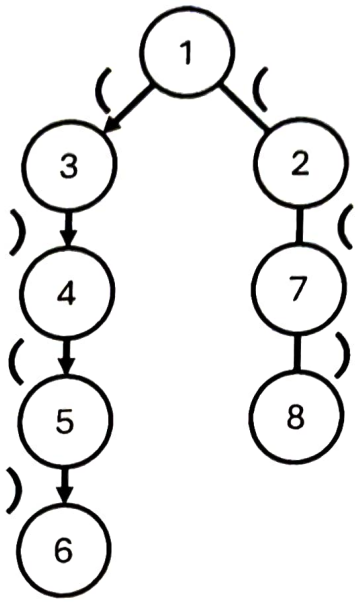
Test 1: the optimal (and only) path is $1 \to 2 \to 3 \to 4 \to 5$, with string "(())", which is valid, so Alice wins.



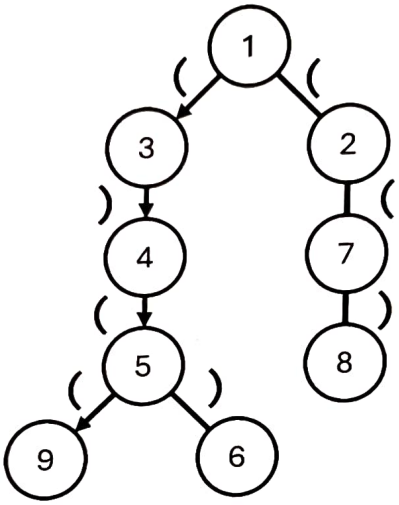
Test 2: the optimal (and only) path is $1 \to 2 \to 3 \to 4 \to 5$, with string "(()(", which is **not** valid, so Bob wins.



Test 3: the optimal path is $1 \to 3 \to 4 \to 5 \to 6$, with string "()()", which is valid, so Alice wins.



Test 4: the optimal path is $1 \to 3 \to 4 \to 5 \to 9$, with string "()((", which is **not** valid, so Bob wins.



Problem B. Josephus Permutation Inversions

Input file:	`standard input`
Output file:	`standard output`
Balloon Color:	`Navy Blue`

The Josephus problem with step size 2 is defined as follows.

There are n people numbered $1, 2, \ldots, n$ standing in a circle.

Starting from person 1, we repeatedly count every second person: skip one person and eliminate the next.

The elimination continues until no one remains.

The order in which people are eliminated forms a permutation of $\{1, 2, \ldots, n\}$, which we call the Josephus permutation j_n. For example, $j_4 = [2, 4, 3, 1]$.

An **inversion** in a permutation $p = [p_1, p_2, \ldots, p_n]$ is a pair of indices (i, j) with $i < j$ and $p_i > p_j$.

The total number of inversions is denoted $i(p)$.

You are given t test cases, each containing an integer n. For each test case, compute $i(j_n)$.

Input

The first line contains an integer t ($1 \le t \le 10^6$) — the number of test cases.

The first line of each test case contains an integer n ($1 \le n \le 10^9$) — the order of the Josephus permutation needed.

Output

For each test case, output a single integer — the number of inversions.

Example

standard input	standard output
3 3 4 121	1 4 2450

Problem C. Kerosene

Input file: standard input
Output file: standard output
Balloon Color: Gold



Yousef has given you an array a of n non-negative integers.

You can perform the following operation any number of times:

- Choose an index i ($1 \le i < |a|$).
- If popcount$(a_i \oplus a_{i+1})$ is **even**, you may replace the two elements a_i and a_{i+1} with a single element $a_i \oplus a_{i+1}$. The remaining elements keep their order, and the length of the array decreases by 1.

Here $\oplus$ denotes the bitwise XOR operation, and popcount(x) is the number of 1-bits in the binary representation of x. For example, popcount$(5) = 2$ because $5 = 101_2$ has two 1-bits in its binary representation. Note that popcount$(0) = 0$, which is even.

Your task is to find the *minimum* possible length of the array after performing any number of operations.

Input

The first line contains an integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \le n \le 2 \times 10^5$) — the number of elements in a.

The second line of each test case contains n integers $a_1, a_2, \ldots, a_n$ ($0 \le a_i \le 10^9$) — the array a.

It is guaranteed that the sum of n over all test cases does not exceed 2×10^5.

Output

For each test case, output a single integer — the minimum possible length of the array after performing any number of operations.

Example

standard input	standard output
3 8 3 1 4 1 5 9 2 6 5 1 2 4 8 16 4 7 7 7 7	5 2 1

Note

In the first test case, the length of the array can be reduced to 5 as follows:

- $[3, 1, 4, 1, 5, 9, 2, 6] \xrightarrow{i=2} [3, 5, 1, 5, 9, 2, 6]$
- $[3, 5, 1, 5, 9, 2, 6] \xrightarrow{i=4} [3, 5, 1, 12, 2, 6]$
- $[3, 5, 1, 12, 2, 6] \xrightarrow{i=1} [6, 1, 12, 2, 6]$

It can be shown that 5 is the minimum possible answer.

Problem D. Last Ride in Doha

Input file:	standard input
Output file:	standard output
Balloon Color:	Black

Ahmed plans to visit $n+1$ stations in a fixed order. Traveling from station i to station $i+1$ costs a_i QAR from his travel card.

Initially, Ahmed has s QAR on his card. At station i, before paying for the next trip, an unused promotional credit worth r_i QAR becomes available. When he uses such a credit, exactly r_i QAR is added to his balance. A credit can be used at most once.

Ahmed may keep an available credit for later. Therefore, before trip i, he may use any unused credit from stations 1 through i.

To pay for trip i, Ahmed must have at least a_i QAR. After the payment, exactly a_i QAR is deducted.

Find the minimum number of promotional credits Ahmed must use to complete all n trips. If it is impossible, print -1.

Input

The first line contains two integers n and s ($1 \le n \le 2 \times 10^5$, $0 \le s \le 10^{18}$) — the number of trips and the initial balance.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^9$) — the trip costs.

The third line contains n integers $r_1, r_2, \ldots, r_n$ ($0 \le r_i \le 10^9$) — the promotional credit values.

Output

Print a single integer — the minimum number of credits Ahmed must use. If completing the journey is impossible, print -1.

Examples

standard input	standard output
5 6 3 5 2 6 1 2 7 0 4 0	2
3 2 4 1 1 1 0 0	-1

Problem E. Lusail Lights

Input file: standard input
Output file: standard output
Balloon Color: Purple

There are n decorative lights arranged from left to right. Their states are described by a binary string s of length n: $s_i = 0$ means that light i is off, and $s_i = 1$ means that it is on.

There are $n-1$ buttons. Pressing button i toggles lights i and $i+1$. Each button may be pressed at most once.

Find the minimum number of presses needed to turn every light off. If it is impossible, print -1.

Input

The first line contains an integer n ($1 \le n \le 2 \times 10^5$).

The second line contains a binary string s of length n.

Output

Print a single integer — the minimum number of presses. If it is impossible, print -1.

Examples

standard input	standard output
4 1100	1
5 10101	-1
6 011110	2

Problem E. Parade Formation

Input file: standard input
Output file: standard output
Balloon Color: Pink

There are n performers at distinct integer positions $x_1, x_2, \ldots, x_n$ on a line. The positions are given in arbitrary order.

They must finish at exactly n consecutive integer positions $p, p+1, \ldots, p+n-1$ for some integer p. Every final position must contain exactly one performer. Moving a performer by one unit costs one unit of effort.

Find the minimum possible total effort.

Input

The first line contains an integer n ($1 \le n \le 2 \times 10^5$).

The second line contains n distinct integers $x_1, x_2, \ldots, x_n$ ($-10^9 \le x_i \le 10^9$).

Output

Print a single integer — the minimum possible total effort.

Examples

standard input	standard output
3 3 2 1	0
4 1 11 20 10	16
5 13 1 4 7 10	12

Problem G. Qatar National Day Flags

Input file: standard input
Output file: standard output
Balloon Color: Green

Nour wants to place n flags in a row. Every flag must be either maroon or white, and no two adjacent flags may have the same color.

Nour may choose either color for the first flag. She may use at most m maroon flags, while the number of white flags available is unlimited.

Determine whether she can place all n flags.

Input

The first line contains two integers n and m ($1 \le n \le 10^9$, $0 \le m \le 10^9$).

Output

Print YES if Nour can place all n flags; otherwise, print NO.

Examples

standard input	standard output
7 3	YES
8 3	NO
1 0	YES

2,4

m w m w m

G → E Test

Problem H. Rest Guys

Input file: standard input
Output file: standard output
Balloon Color: Sand White

Boggy's brothers own a massive factory that operates on a continuous schedule. Today, the factory needs to execute n specific tasks.

For the i-th task, exactly k_i workers are required. The task starts at hour o_i and lasts for p_i consecutive hours. All tasks are guaranteed to finish within the same day, so $o_i + p_i \leq 24$.

The factory employs exactly z workers. Each worker has a total capacity of 24 hours during the day. However, the j-th worker must rest for at least r_j hours.

Workers are completely interchangeable. They may be swapped in and out of tasks at any time, and a worker's rest hours do not have to be consecutive.

Given the schedule of all tasks and the rest requirements of all workers, determine whether it is possible to assign workers to the tasks such that every task has the required number of workers at every hour and every worker gets at least the required amount of rest.

Input

The first line contains two integers n and z ($1 \leq n, z \leq 10^5$) — the number of tasks and the number of workers.

The second line contains n integers $p_1, p_2, \ldots, p_n$ ($1 \leq p_i \leq 24$) — the duration of each task in hours.

The third line contains n integers $o_1, o_2, \ldots, o_n$ ($0 \leq o_i \leq 23$) — the starting hour of each task.

It is guaranteed that $o_i + p_i \leq 24$ for every task.

The fourth line contains n integers $k_1, k_2, \ldots, k_n$ ($1 \leq k_i \leq z$) — the number of workers required for each task.

The fifth line contains z integers $r_1, r_2, \ldots, r_z$ ($0 \leq r_j \leq 24$) — the minimum number of rest hours required by each worker.

Output

Print `YES` if it is possible to complete all tasks while ensuring that every worker gets at least their required number of rest hours; otherwise, print `NO`.

Example

standard input	standard output
2 3 4 5 8 14 1 2 8 8 8	YES

Problem I. Can you see it?

Input file:	`standard input`
Output file:	`standard output`
Balloon Color:	`Red`

Suppose you live on an infinite line.

There is a beautiful building you cannot take your eyes off, located at position x_0 with height h_0. There are n ugly buildings on both sides of the beautiful building. Each ugly building is strictly shorter than the beautiful building.

You wonder whether you can still see the beautiful building from a given point (x, h).

Each building is a vertical segment from the ground up to its height. You can see the beautiful building if the segment from your point (x, h) to the top of the beautiful building does not touch any ugly building.

Note that if an ugly building lies exactly on the line of sight, it still blocks the view.

Input

The first line contains an integer t ($1 \le t \le 5000$) — the number of test cases.

The first line of each test case contains two integers n and q ($1 \le n, q \le 2 \cdot 10^5$) — the number of ugly buildings and the number of queries.

The second line of each test case contains two integers x_0 and h_0 — the position and height of the beautiful building.

The next n lines contain two integers x_i and h_i ($1 \le i \le n$) — the position and height of the i-th ugly building.

The next q lines contain two integers x and h ($-10^6 \le x \le 10^6$, $1 \le h \le 10^6$) — a point from which you want to check whether the beautiful building is visible.

It is guaranteed that $-10^6 \le x_0, x_i \le 10^6$, $2 \le h_0 \le 10^6$, and $1 \le h_i \le 10^6$. Moreover, $h_i < h_0$ for every ugly building, no two buildings share the same position, no query point coincides with any building, and the sum of $n + q$ over all test cases does not exceed $3 \cdot 10^6$.

Output

For each query, output `YES` if you can see the beautiful building; otherwise, output `NO`.

Examples

standard input	standard output
1 3 2 10 10 1 3 2 4 3 1 -1 5 -2 3	YES YES
1 1 2 10 10 5 5 0 6 2 2	YES NO
1 3 2 10 10 1 3 2 4 3 1 11 5 12 3	YES YES

Problem J. Space Traversal

Input file: `standard input`
Output file: `standard output`
Balloon Color: `Yellow`

You are exploring an n-dimensional space. You start at the origin $o = (0, 0, \ldots, 0)$, and your goal is to reach the destination point $a = (a_1, a_2, \ldots, a_n)$.

In a single step, if your current position is $x = (x_1, x_2, \ldots, x_n)$, you can move to a new position $y = (y_1, y_2, \ldots, y_n)$ if and only if the following two conditions are met:

- $y_i \geq x_i$ for all $1 \leq i \leq n$.
- $y \neq x$.

A valid path is a sequence of points $p_0, p_1, \ldots, p_k$ such that $p_0 = o$, $p_k = a$, and the transition from p_{j-1} to p_j is a valid step for all $1 \leq j \leq k$.

All coordinates of every point on a valid path must be integers.

Find the total number of distinct valid paths from the origin to the destination point a. Two paths are considered distinct if they have a different number of steps or visit a different point at any step.

Since the number of paths can be very large, output the answer modulo $10^9 + 7$.

Input

The first line contains an integer n ($1 \leq n \leq 5 \times 10^4$).

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($0 \leq a_i \leq 5 \times 10^4$).

It is guaranteed that $1 \leq \sum_{i=1}^{n} a_i \leq 5 \times 10^4$.

Output

Print a single integer — the total number of valid paths from the origin to a modulo $10^9 + 7$.

Examples

standard input	standard output
2 1 1	3
3 4 2 3	51648

Problem K. The Glitched Ultimate

Input file: standard input
Output file: standard output
Balloon Color: Orange

You are playing a new video game and trying to achieve the "Perfect High Score" to win the global tournament. In the final level, you face n monsters. The i-th monster has h_i health points (HP).

To maximize your score, you must defeat all the monsters using the minimum number of attacks. You have two types of attacks available, and each uses exactly one turn:

1. **Basic Attack:** Target a single monster that is still alive and reduce its HP by 1.
2. **The Glitched Ultimate:** An area-of-effect (AoE) attack that reduces the HP of all monsters by 1. However, there is a known bug in the game: every monster whose current HP is equal to the maximum current HP glitches out, evading the Ultimate attack and taking no damage.

Important: If there are multiple monsters tied for the maximum HP, all of them evade the attack, and their HP remains unchanged.

A monster is considered defeated when its HP drops to 0 or below.

What is the minimum number of attacks required to defeat all monsters and secure the Perfect High Score?

Input

The first line contains an integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \le n \le 10^5$) — the number of monsters.

The second line of each test case contains n integers $h_1, h_2, \ldots, h_n$ ($1 \le h_i \le 10^9$) — the initial HP of the monsters.

It is guaranteed that the sum of n over all test cases does not exceed 10^5.

Output

For each test case, output a single integer — the minimum number of attacks required to defeat all monsters.

Example

standard input	standard output
3 3 5 2 1 4 5 5 5 2 2 100 100	7 11 200

Problem L. Walk on tree

Input file: `standard input`
Output file: `standard output`
Balloon Color: `Blue`

You are given a rooted tree with n vertices, rooted at vertex 1, and a string s consisting only of the uppercase letters `'U'` and `'D'`.

Omar_Farhan is playing a game on this tree.

Before performing the moves, Omar chooses a starting node and a single root-to-leaf path that contains this starting node.

He must perform all moves while staying on this chosen path.

For each character of the chosen substring of s:

- If the current character is `'U'`, Omar moves to the parent of the current node along the chosen path.
- If the current character is `'D'`, Omar moves to the child of the current node along the chosen path.

If at any point the required move does not exist on the chosen path, Omar loses the game.

For every query, Omar may choose both the starting node and the root-to-leaf path in a way that helps him complete the required sequence of moves.

You are given q queries. Each query contains two integers l and r.

For each query, determine the number of starting nodes for which Omar can choose a root-to-leaf path containing that node and successfully perform all moves in $s_l s_{l+1} \dots s_r$ without losing.

Input

The first line contains an integer n ($1 \le n \le 10^5$) — the number of vertices in the tree.

The next $n-1$ lines contain two integers u and v ($1 \le u, v \le n$, $u \ne v$) — an undirected edge between vertices u and v.

The tree is rooted at vertex 1.

The next line contains the string s ($1 \le |s| \le 10^5$), consisting only of the characters `'U'` and `'D'`.

The next line contains an integer q ($1 \le q \le 10^5$) — the number of queries.

The next q lines contain two integers l and r ($1 \le l \le r \le |s|$).

Output

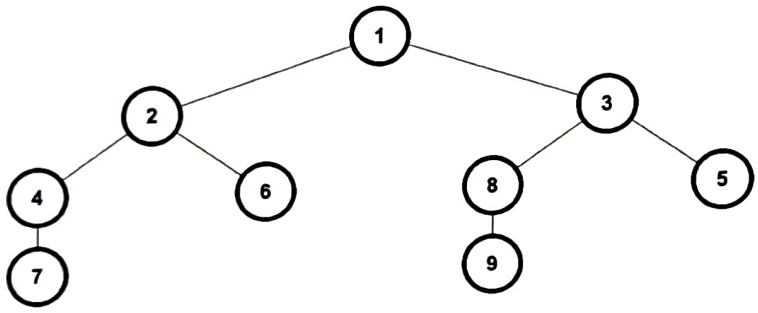
For each query, output a single integer — the number of starting nodes for which Omar can choose a root-to-leaf path containing that node and successfully perform all moves in $s_l s_{l+1} \dots s_r$ without losing.

Example

standard input	standard output
9 1 2 1 3 2 4 2 6 3 8 3 5 4 7 8 9 UDDDUDUUDUU 5 1 3 2 6 5 11 4 10 7 9	4 1 2 4 6

Note

The tree in the sample is:



The string is $s =$ UDDDUDUUDUU. For each query, the valid starting nodes are:

- $[1, 3]$: UDD works from nodes $\{2, 3, 4, 8\}$, so the answer is 4.
- $[2, 6]$: DDDUD works only from the root, node $\{1\}$, so the answer is 1.
- $[5, 11]$: UDUUDUU works from the two deepest leaves, nodes $\{7, 9\}$, so the answer is 2.
- $[4, 10]$: DUDUUDU works from nodes $\{2, 3, 4, 8\}$, so the answer is 4.
- $[7, 9]$: UUD works from every node at depth at least 2, namely $\{4, 5, 6, 7, 8, 9\}$, so the answer is 6.

The five answers are therefore 4, 1, 2, 4, and 6.